Experiment Number : 8 **Date: 20.02.26**

Implementation of Packages in Java

Aim:

Write a Java program to implement built-in, user-defined packages and accessing all classes in a package.

PRE LAB EXERCISE:

QUESTIONS:

1. **What is java.util package and what collection framework does it contain?**

   - Java.util is a built-in Java package that provides utility classes and data structures.
   - It contains the Collection Framework, which includes:
     • List (ArrayList, LinkedList)
     • Set (HashSet, TreeSet)
     • Queue (PriorityQueue)
     • Map (HashMap, TreeMap)

2. **What are the two types of packages in Java?**

   The two types of packages in Java are:

   • Built-in (Predefined) packages – Provided by Java (example: java.util, java.io).

   • User-defined packages – Created by the programmer

IN LAB EXERCISE

Objective

To understand and implement the concepts of built-in, user-defined packages and accessing all classes in a package in Java.

Built-in Packages comprise a large number of classes that are part of the Java API. Some of the commonly used built-in packages are:

- java.lang: Contains language support classes(e.g, classes that define primitive data types, math operations). This package is automatically imported.
- java.io: Contains classes for supporting input/output operations.
- java.util: Contains utility classes that implement data structures such as Linked Lists and Dictionaries, as well as support for date and time operations.
- java.applet: Contains classes for creating Applets.
- java.awt: Contains classes for implementing the components for graphical user interfaces (like buttons, menus, etc).

Source Code

```
import java.util.Random;   // built-in package

public class Sample{

      public static void main(String[] args) {

        // using Random class

       Random rand = new Random();

       // generates a number between 0–99

       int number = rand.nextInt(100);

       System.out.println("Random number: " + number);

   }

}
```

Output

```
Random number: 16
```

User-defined Packages are the packages that are defined by the user.

Source code

```
package com.myapp;

public class Helper {

   public static void show() {

       System.out.println("Hello from Helper!");
```

```
    }
}
```

==To use this in another class==

```
import com.myapp.Helper;
public class Test {
    public static void main(String[] args) {
        Helper.show();
    }
}
```

Output:

```
Hello from Helper!
```

//Importing all classes from a package.

Source code

```
import java.util.Vector;
public class Coders {
      public Coders() {
            // java.util.Vector is imported, We are able to access it directly in our code.
        Vector v = new Vector();
        java.util.ArrayList l = new java.util.ArrayList();
        l.add(3);
        l.add(5);
        l.add(7);
        System.out.println(l);
    }
    public static void main(String[] args) {
```

```
        new Coders();
    }
}
```

Output

```
[3,5,7]
```

POST LAB EXERCISE

1. **What will happen if two classes in different packages have the same name and are imported in a Java file?**

   - If two classes with the same name are imported, Java gives a compile-time error (ambiguity error).
   - To solve it, we must use the fully qualified class name.

2. **What is the purpose of using packages in Java?**

   Packages are used to:
   • Organize related classes together
   • Avoid name conflicts
   • Improve code reusability
   • Provide access protection (security)
   • Make large projects manageable
   In short:
   Packages help in organization, security, and avoiding naming conflicts.

3. **Which built-in Java package would you use if you want to create a GUI window and display a message?**
   A. **java.util**
   B. **java.sql**
   C. **java.awt**
   D. **java.net**

Correct answer:

C. java.awt

java.awt is used to create GUI components like windows, buttons, and labels.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		

Experiment Number : 9 **Date: 20.02.26**

Implementation of a Java Program to import packages using different methods

Aim:

Write a Java program to import packages using different methods for different use cases.

PRE LAB EXERCISE

QUESTIONS

1. **How to import a single class and multiple classes from a package in Java?**

   Import a single class:
   import java.util.ArrayList;
   Import multiple classes (entire package):
   import java.util.*;

2. **Which package is always imported by default in every Java class?**

   - java.lang
   - It is automatically imported and contains commonly used classes like String, System, Math, Integer, etc.

IN LAB EXERCISE

Objective

To understand and implement the Java packages using different methods and import them.

Problem

Define a package named 'useFul' with a class names 'UseMe' having following methods:

1) area()- To calculate the area of given shape.

2) salary()- To calculate the salary given basic Salary,da,hRA.

3) percentage()-To calculate the percentage given total marks and marks obtained.

4) Develop a program named 'Package Use' to import the above package 'useFul' and use the method area().

5) Develop a program named ‘manager’

Source Code

```
//Package Creation:
package useFull;
import java.util.*;
public class UseMe
{
        Scanner obj=new Scanner(System.in);
public static void area()
      {
          class method{
           void aos(int a)
           {
             System.out.print("\nArea of square with length "+a+" is "+(a*a));
           }
          void aor(int a,int b)
          {
             System.out.print("\nArea of reactangle with dimensions "+a+" & "+b+" is "+(a*b));
           }
           void aoc(int r)
          {
            double a=3.14*r*r;
           }
          System.out.print("\nArea of circle with radius "+r+" is "+a);
        }
        void aot(int a,int b)
         {
          float ar=(a*b)/2;
```

```
    System.out.print("\nArea of triangle with dimensions "+a+" &"+b+" is "+ar);
  } }
Scanner obj=new Scanner(System.in);
method m=new method();
System.out.print("\n1.Square\n2.Rectangle\n3.Circle\n4.Triangle\nSelect the shape\n");
int ch=obj.nextInt();
UseMe u=new UseMe();
switch(ch)
        {
        case 1:System.out.print("\nEnter the length of side of square : ");
                int s=obj.nextInt();m.aos(s);
                break;
        case 2:System.out.print("\nEnter the dimensions of rectangle : ");
                int l=obj.nextInt();
                int b=obj.nextInt();
                m.aor(l,b);
                break;
        case 3:System.out.print("\nEnter the radius of circle : ");
                int r=obj.nextInt();
                m.aoc(r);
                break;
        case 4:System.out.print("\nEnter the dimensions of triangle : ");
                int ba=obj.nextInt();
                int w=obj.nextInt();
                m.aot(ba,w);
                break; } }
public void salary()
  {
```

```
        int ba,da,hra;
        System.out.print("\nEnter the basic salary : ");
        ba=obj.nextInt();
        System.out.print("\nEnter the dearness allowance :");
        da=obj.nextInt();
        System.out.print("\nEnter the house rent allowance : ");
        hra=obj.nextInt();
        System.out.print("\nThe total Gross salary of employee is : "+(ba+da+hra));
    }
public void percentage()
    {
        int n,sum=0;
        float p;
        System.out.print("\nEnter the total number of subjects : ");
        n=obj.nextInt();
        int m[]=new int[n];
        System.out.print("\nEnter the marks of "+n+" subjects : ");
        for(int i=0;i<n;i++)
        {
            m[n]=obj.nextInt();
        }
        for(int i=0;i<n;i++)
        {
             sum=sum+m[i];
        }
        p=sum/n;
        {
                System.out.print("\nPercentahe of student : "+p);
```

```
        }
    }
}
```

//Package Implementation-1:

```
        import useFull.UseMe;
        class packageUse
        {
                public static void main(String args[])
                {
                UseMe o=new UseMe();o.area();
                }
        }
```

Output

```
1.      Square
2.      Rectangle
3.      Circle
4.      Triangle
Select the shape
1

Enter the length of side of square:
4
Area of the square with length 4 is 16
```

//Package Implementation-2:

```
        import useFull.UseMe;
        class manager
        {
           public static void main(String args[])
           {
```

```
        UseMe obj=new UseMe();obj.salary();
        }
    }
```

Output

```
Enter the basic salary:
40000
Enter the dearness allowance:
1000
Enter the house rent allowance:
7000
The total Gross salary of employee is: 48000
```

POST LAB EXERCISE

1. **Find the key differences between java.util and java.lang packages.**

Feature	**java.lang**	**java.util**
Import Requirement	Automatically imported	Must be imported manually
Purpose	Provides fundamental classes	Provides utility classes and data structures
Main Usage	Core language support	Collections, utilities, date/time, etc.
Examples of Classes	String, System, Math, Integer	Scanner, ArrayList, HashMap, Date

2. **List some of the subpackages of java.util**

   Some important subpackages of java.util are:
   - java.util.concurrent → For multithreading utilities
   - java.util.function → Functional interfaces (Java 8)
   - java.util.stream → Stream API for data processing
   - java.util.logging → Logging framework
   - java.util.regex → Regular expression support

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		

Experiment Number : 10 **Date: 21.02.26**

Implementation of Access Modifiers in Java

Aim:

Write a Java program to implement different access modifiers.

PRE LAB EXERCISE

QUESTIONS

1. **What are the 4 access modifiers available in Java?**

Access Modifier	Accessibility
private	Only within the same class
default (no keyword)	Within the same package
protected	Same package + subclasses (even in different packages)
public	Accessible from anywhere

2. **Which access modifiers can be used in Java to control access to class members?**

   In Java, all four access modifiers can be used to control access to class members (variables, methods, constructors):

- **private** – Accessible only within the same class.
- **default** (no keyword) – Accessible within the same package.
- **protected** – Accessible within the same package and by subclasses in other packages.
- **public** – Accessible from anywhere.

  These modifiers help in achieving **encapsulation** and controlling data security.

IN LAB EXERCISE:

Objective

To demonstrate different access modifiers such as Default, Private, Protected and Public using Java programs.

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Fig: Comparison table of Access Modifiers in Java

Source Code

// Default Access modifier

```
class Car {
   String model; // default access
}
public class Main {
      public static void main(String[] args){
       Car c = new Car();
       c.model = "Tesla"; // accessible within the same package
       System.out.println(c.model);
   }
}
```

Output:

```
<terminated> exp10 [Java Application] C:\Users\Shri Sanjanaa\.p
Tesla
```

//Private Access Modifier

```
class Person {
    // private variable
    private String name;
    public void setName(String name)  {
     this.name = name; // accessible within class
    }
    public String getName() {
            return name;
   }
}
public class Geeks {
    public static void main(String[] args)
    {
       Person p = new Person();
        p.setName("Alice");

        // System.out.println(p.name);
        // Error: 'name'
        // has private access
        System.out.println(p.getName());
    }
}
```

Output

```
<terminated> exp10 [Java Application] C:\Users
Alice
```

//Protected Access Modifier

```
class Vehicle {
   protected int speed; // protected member
}
class Bike extends Vehicle {
   void setSpeed(int s)
   {
      speed = s; // accessible in subclass
   }
   int getSpeed()
   {
      return speed; // accessible in subclass
   }
}
public class Main {
   public static void main(String[] args){
      Bike b = new Bike();
      b.setSpeed(100);
      System.out.println("Access via subclass method: "+ b.getSpeed());
      Vehicle v = new Vehicle();
      System.out.println(v.speed);
   }
}
```

Output

```
<terminated> Main [Java Application] C:\Users\Shri
Access via subclass method: 100
0
```

//Public Access Modifier

```
class MathUtils {
        public static int add(int a, int b) {
        return a + b;
    }
}
public class Main {
        public static void main(String[] args) {
        System.out.println(MathUtils.add(5, 10)); // accessible anywhere
    }
}
```

Output

```
<terminated> Main [Java Application] C:\
15
```

POST LAB EXERCISE

1. **Write a Java program to implement Default access modifier which has a default class within the same package and a default class from a different package.**

   In Java, default access modifier (no keyword) allows access only within the same package. It cannot be accessed from a different package.

   Same Package (Accessible)
   Package: pack1

```
package pack1;

class DefaultClass {   // default access class
    void display() {
        System.out.println("Inside Default Class");
    }
}
package pack1;
```

```
public class Main1 {
    public static void main(String[] args) {
        DefaultClass obj = new DefaultClass();
        obj.display();   // Accessible (same package)
    }
}
```

Output:
Inside Default Class

Different Package (Not Accessible)
If another class in a different package tries to access DefaultClass, it will give a compilation error, because default classes are not visible outside their package.

Conclusion:
Default access modifier allows access only within the same package and restricts access from other packages.

2. Which access modifier provides the highest level of access?

In Java, the public access modifier provides the highest level of access.
A public class, method, or variable can be accessed from:

- Any class
- Any package
- Anywhere in the program

It has no access restrictions.
Access level order (lowest to highest):
private → default → protected → public

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	

Experiment Number :11 **Date: 24.02.26**

Implementation of multiple inheritance in Java using Interface

Aim:

Write a Java program to implement multiple inheritance using Interface.

PRE LAB EXERCISE

QUESTIONS

1.Why Java does not support multiple inheritance using classes like that of C?

- Java does not support multiple inheritance using classes to avoid the **Diamond Problem**, which creates ambiguity when two parent classes have the same method.
- If a child class inherits from two classes that both inherit from the same superclass, the compiler gets confused about which method to execute.
- To avoid this ambiguity and keep the language simple and secure, Java allows multiple inheritance only through **interfaces**, not classes.

2.What is the primary purpose of an interface in Java?

A. To store data using instance variables

B. To define a contract of methods a class must implement

D. To allow creation of objects

D. To improve runtime performance

Correct Answer: B. To define a contract of methods a class must implement

- An interface in Java defines method declarations without implementation. Any class that implements the interface must provide definitions for those methods.
- Thus, an interface acts as a contract that ensures consistency and abstraction.

IN LAB EXERCISE

Objective

To demonstrate how an interface in Java defines constants and abstract methods, which are implemented by a class.

Source Code

```
import java.io.*;
  // Add interface
interface Add{
    int add(int a,int b);
  }
// Sub interface
interface Sub{
            int sub(int a,int b);
  }
// Calculator class implementing Add and Sub
class Cal implements Add , Sub
  {
```

```
// Method to add two numbers
        public int add(int a,int b){
        return a+b;
    }
        // Method to sub two numbers
        public int sub(int a,int b){
        return a-b;
    }
}
class Example{
    // Main Method
    public static void main (String[] args){

        // instance of Cal class
        Cal x = new Cal();
        System.out.println("Addition : " + x.add(2,1));
        System.out.println("Substraction : " + x.sub(2,1));
    }
}
```

Output:

```
<terminated> exp11 [Java Application]
Addition : 3
Substraction : 1
```

POST LAB EXERCISE

1. **Can a functional interface extend another interface?**

   - **Yes,** a functional interface can extend another interface, but it must still have only one abstract method in total.
   - If it extends an interface that already has one abstract method, it cannot add another abstract method, otherwise it will no longer remain functional.

   Example:

```
interface A {
   void show();
}

@FunctionalInterface
interface B extends A {
   // No new abstract method added
}
```

   Here, `B` is still a functional interface because there is only **one abstract method** (`show()`).

2. **Which feature was introduced in interfaces starting from Java 8?**

   A. Constructors

   B. Private methods

   C. Default and static methods

   D. **Final classes**

Correct Answer: C. Default and static methods

Explanation:

Before Java 8, interfaces could contain only:

- Abstract methods
- Constants

From Java 8, interfaces can also contain:

- Default methods
- Static methods

These allow method implementation inside interfaces.

ASSESSMENT :

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	

Experiment Number : 12 **Date: 25.02.26**

Implementation of exception handling in Java

Aim:

Write a java program to implement exception handling.

PRE LAB EXERCISE

QUESTIONS

1.What is exception handling in Java?

- Exception handling in Java is a mechanism used to handle runtime errors so that the normal flow of the program is not interrupted.
- An exception is an unexpected event that occurs during program execution (like dividing by zero or accessing invalid array index).
- Java handles exceptions using keywords like:
  try, catch, finally, throw, throws.
- Thus, exception handling helps maintain program stability and prevents abnormal termination.

2.What is the purpose of using try-catch blocks in exception handling?

The **try-catch block** is used to handle exceptions safely.

- The **try block** contains the code that may cause an exception.
- The **catch block** handles the exception if it occurs.

Purpose:

- To prevent program crash.
- To handle errors gracefully.
- To continue execution after handling the error.

IN LAB EXERCISE

Objective

To understand and implement exception handling through try-catch and finally blocks.

Source Code

```
import java.util.InputMismatchException;
import java.util.Scanner;
public class ExceptionHandlingExample {
 public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
         try {
        System.out.print("Enter an integer: ");
        int num = scanner.nextInt();
         int result = 10 / num;
        System.out.println("Result: " + result);
        } catch (InputMismatchException e) {
        System.out.println("Invalid input! Please enter an integer.");
        } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero!");
        } finally {
        scanner.close();
        System.out.println("Program execution completed.");
        } }
}
```

Outputs

Output 1 (Valid Input - Non-zero Integer):

```
<terminated> exp11 [Java Application] C:\Users\Shri Sanjanaa\
Enter an integer: 5
Result: 2
Program execution completed.
```

Output 2 (Valid Input - Zero):

```
<terminated> exp11 [Java Application] C:\Users\Shri Sanjanaa\
Enter an integer: 0
Cannot divide by zero!
Program execution completed.
```

Output 3 (Invalid Input - Non-integer):

```
<terminated> exp11 [Java Application] C:\Users\Shri Sanjanaa\.p2
Enter an integer: abc
Invalid input! Please enter an integer.
Program execution completed.
```

POST LAB EXERCISE

1.What is the purpose of the Scanner object in the Java program?

The **Scanner** object is used to take input from the user during program execution.

It reads different types of data such as:

- Integers (nextInt())
- Floating numbers (nextDouble())
- Strings (nextLine())
- Characters (using string methods)
  It is mainly used to get input from:
- Keyboard (System.in)
- Files
- Other input streams

2.What exceptions are expected to be thrown in the code within the try block? Why?

The expected exceptions depend on the code inside the try block. Common exceptions when using Scanner and arithmetic operations are:

1. **InputMismatchException:**
   - Occurs if the user enters wrong data type.

    - Example: Entering a string when nextInt() expects an integer.
2. **ArithmeticException:**
    - Occurs when dividing a number by zero.
3. **NumberFormatException:**
    - Occurs when converting an invalid string into a number.

These exceptions occur because:

- User input may be invalid.
- Arithmetic operations may involve illegal operations like division by zero.
- Data conversion may fail.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	

Experiment Number : 13 **Date: 26.02.26**

Implementation of User-defined Exception in Java

Aim:

Write a java program to implement User-defined Exception.

PRE LAB EXERCISE

QUESTIONS

1.What is Java custom exception?

- A **custom exception** in Java is a user-defined exception created by extending the Exception or RuntimeException class.
- It is used to handle application-specific errors that are not covered by Java's built-in exceptions.
- Programmers create custom exceptions to define meaningful error messages for specific conditions in a program.

2.What are the two types of custom exceptions in Java?

There are two types of custom exceptions:

Checked Custom Exception

- Extends Exception class.
- Must be handled using try-catch or declared using throws.
- Checked at compile time.

Unchecked Custom Exception

- Extends RuntimeException class.
- Not mandatory to handle.
- Checked at runtime.

IN LAB EXERCISE

Objective

To understand and implement Checked and Unchecked custom exceptions.

Source Code

```
Ex 1: // Custom Checked Exception
class InvalidAgeException extends Exception {
   public InvalidAgeException(String m) {
      super(m);
   }
}
// Using the Custom Exception
public class AgeCheck{
   public static void validate(int age)
     throws InvalidAgeException {
      if (age < 18) {
         throw new InvalidAgeException("Age must be 18 or above.");
      }
      System.out.println("Valid age: " + age);
   }
public static void main(String[] args) {
  try {
         validate(12);
      } catch (InvalidAgeException e) {
         System.out.println("Caught Exception: " + e.getMessage());
      }
   }
}
```

Output 1 :

```
<terminated> exp11 [Java Application] C:\Users\Shri Sanjanaa\.
Caught Exception: Age must be 18 or above.
```

Ex 2: // Custom Unchecked Exception

```
class DivideByZeroException extends RuntimeException {
   public DivideByZeroException(String m) {
      super(m);
   }
}
// Using the Custom Exception
public class TestSample {
   public static void divide(int a, int b) {
      if (b == 0) {
         throw new DivideByZeroException("Division by zero is not allowed.");
      }
      System.out.println("Result: " + (a / b));
   }
public static void main(String[] args) {
  try {
         divide(10, 0);
      } catch (DivideByZeroException e) {
         System.out.println("Caught Exception: " + e.getMessage());
      }
   }
}
```

Output 2:

```
<terminated> exp11 [Java Application] C:\Users\Shri Sanjanaa\.p2\pool\
Caught Exception: Division by zero is not allowed.
```

POST LAB EXERCISE:

1.What is required to create a custom exception in Java?

A. **Extend the Object class**
B. **Extend the Throwable class**
C. **Extend Exception or RuntimeException**
D. **Implement the Serializable interface**

Correct Answer: C. Extend Exception or RuntimeException

To create a custom exception in Java, we must create a class that extends either:

- Exception → for checked exception
- RuntimeException → for unchecked exception

2.List some use cases for the checked and unchecked exceptions.

Checked Exceptions (Compile-time exceptions)

These must be handled using try-catch or declared using throws.

Use Cases:

- File handling errors (IOException)
- Database connection errors (SQLException)
- Network-related errors
- User input validation in critical systems
- Reading/writing files

Used when the error is **recoverable** and the programmer is expected to handle it.

Unchecked Exceptions (Runtime exceptions)

These occur at runtime and handling is optional.

Use Cases:

- Dividing by zero (ArithmeticException)
- Accessing invalid array index (ArrayIndexOutOfBoundsException)
- Null reference access (NullPointerException)
- Logical programming errors

  Used when the error is due to **programming mistakes**.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	